

C++&Qt Day01 hw3 AI 보고서

작성자: Doyeon Park

작성일: 2026.09.03

목차

1. AI 사용 목적
2. AI 를 사용한 부분
3. 깨달은 점

1. AI 사용 목적

이전에 한 hw1, hw2 와 달리 Player 과 Monster 두 클래스를 이용하여 프로그램을 작성하는 부분에서 어려움을 느껴 코드 구조를 설계할때 AI 를 활용했습니다. 또한 클래스 간 상호작용 부분이 어려워 이와 관련된 부분을 AI 의 도움을 받아 코드를 작성했습니다. 이 외에도 코드 오류나 검토에서 AI 를 사용하였습니다.

2. AI 를 사용한 부분

1단계: 헤더 2개부터

monster.hpp

네가 이런 구조로 직접 작성:

```
</> C++  
  
#ifndef MONSTER_HPP  
#define MONSTER_HPP  
  
class Monster  
{  
public:  
    // HP, x, y  
  
    // 기본 생성자  
    // 매개변수 생성자  
  
    // 공격받는 함수  
};  
  
#endif
```

player.hpp

```
</> C++  
  
#ifndef PLAYER_HPP  
#define PLAYER_HPP  
  
#include "monster.hpp"  
  
class Player  
{  
public:  
    // HP, MP, x, y  
  
    // 기본 생성자  
    // 매개변수 생성자  
  
    // 공격  
    // 상태 출력  
    // X 이동  
    // Y 이동  
};  
  
#endif
```

일단 monster.hpp, player.hpp 두 개 작성해서 보내봐. 이번 과제는 클래스 모양 자체가 PPT에 지정돼 있으니까 거기 맞춰서 가자.

monster 와 player 의.hpp 의 기본 구조 작성에 도움을 받았습니다.

이후 monster.cpp, player.cpp 도 hpp 와 같은 방법으로 순차적으로 작성하던 중 ,

```
void Player::Attack(Monster &target)
{
    MP = MP - 1;

    if (x == target.x && y == target.y)
    {
        cout << "공격 성공!" << endl;
        cout << "남은 체력:" << target.Be_Attacked() << endl;
    }
    else
    {
        cout << "공격 실패!" << endl;
    }
}
```

이 부분에서 target 부분이 생소하여 AI 에게 추가 설명을 요청하였고,

target은 뭐야? monster의 저장공간 위치인가?

거의 맞게 이해했어. `target` 은 `Monster` 객체 그 자체를 가리키는 참조 이름이야.

이 선언을 보면:

```
<> C++
void Player::Attack(Monster &target)
```

여기서:

`Monster`

→ `target` 의 자료형

`&`

→ 복사본이 아니라 원래 객체를 직접 참조

`target`

→ 함수 안에서 그 `Monster` 객체를 부르는 이름

이야.

예를 들어 `main()` 에서:

```
<> C++
Monster monster(5, 4, 50);
player.Attack(monster);
```

라고 하면 `Attack()` 안에서는:

```
<> C++
target
```

이 사실상 `monster` 의 별명처럼 동작해.

`target.Be_Attacked();`
그럼 이게 `Monster` 이름의 class 안에있는 `Be_Attacked`의 함수를 건드리는 거네

맞아. 정확히는 `target` 이 가리키는 `Monster` 객체의 `Be_Attacked()` 멤버함수를 호출하는 거야.

```
<> C++
target.Be_Attacked();
```

들 보면:

```
target
→ 어떤 Monster 객체를 참조하고 있음
.Be_Attacked()
→ 그 Monster 객체의 Be_Attacked() 멤버함수 실행
```

예를 들어:

```
<> C++
Monster monster(5, 4, 50);
player.Attack(monster);
```

그리고 `Attack()` 안이:

```
<> C++
void Player::Attack(Monster &target)
{
    target.Be_Attacked();
}
```

이런 실제로는 이런 느낌이야:

```
<> C++
monster.Be_Attacked();
```

가 실행되는 것과 같은 효과야.

그래서 `Be_Attacked()` 안에서:

위 사진과 같은 추가 대화를 통해 `target` 이 `Monster` 객체를 참조하는 참조 변수라는 것을 이해하게 되었고 코딩을 이어나갈 수 있었습니다.

3. 깨달은 점

이 코드를 짜면서 깨달은 점이 많았던것 같습니다. 먼저 첫번째로 클래스는 각각 따로 존재하지만, 한 클래스의 멤버 함수에서 다른 클래스의 객체를 참조해 동작시킬 수 있다는 점이 흥미로웠습니다. 이 때문에 target.Be_Attacked()를 호출하면 복사본이 아니라 실제 Monster 의 HP 를 변경시킬 수 있었습니다.

또한 this->x=x 를 이용하여 왼쪽은 멤버변수, 오른쪽은 매개변수로 표현할 수 있는 점이 인상깊었습니다.

이후 hw2.cpp 와 main.cpp 도 동일한 방식으로 진행하였다.

예:

```
점의 개수 : 10  
좌표 최소값 : 0  
좌표 최대값 : 20
```

그리고 여기서 점 개수만큼 메모리를 만들자.

아까 정수 배열에서는:

```
C++  
arr = new int[size];
```

했지.

이번에는 `int` 배열이 아니라 `Point` 배열이니까 어떤 식이 될지 생각해봐.

```
C++  
points = new ???[count];
```

이 부분까지 네가 채우면 돼.

즉 이번에 작성할 부분은:

```
C++  
PointDistance::PointDistance()  
{  
    // 초기화  
}  
  
PointDistance::~PointDistance()  
{  
    // 메모리 해제  
}  
  
void PointDistance::input()  
{  
    // count 입력  
  
    // minCoord 입력  
  
    // maxCoord 입력  
  
    // Point 배열 동적할당  
}
```

여기까지만 작성해서 보내줘. 그다음 `generatePoints()` 에서 랜덤 좌표 만드는 법으로 넘어가자.

이제 거의 끝났고, 남은 건 `main.cpp` 야.

이번 흐름은:

```
객체 생성  
↓  
입력  
↓  
랜덤 점 생성  
↓  
점 출력  
↓  
거리 계산  
↓  
결과 출력
```

이니까 `main.cpp` 에서 호출 순서만 맞추면 돼.

그리고 랜덤값이 매번 다르게 나오게 하려면 `main.cpp` 에서:

```
C++  
#include <cstdlib>  
#include <ctime>
```

추가하고,

```
C++  
srand(time(nullptr));
```

를 프로그램 시작할 때 한 번 넣으면 돼.

이제 `main.cpp` 직접 작성해서 보내봐.

📄 🔍 🔄 ⋮

```
#include <cstdlib>  
#include <ctime>  
#include "hw2.hpp"  
  
srand(time(nullptr));  
  
int main(void)  
{  
    PointDistance pointdistance;  
  
    pointdistance.input();  
  
    더 보기 >
```

이후 오타 및 코드 총 검토 또한 AI 에게 맡겼다.

3. 깨달은 점

함수 내용 작성시 Distance 최소 최대 판별할때 이중 for 문을 쓰면서 헷갈렸던 부분이있었다. 이때 `distance[i][j]`처럼 모든 거리를 저장하려 했지만 잘 해결되지 않아 ai 에게 질문했었다. 이후 실제로는 현재 두 점의 거리만 저장해서 바로 최대 최소와 비교하면 된다는것을 깨달았고 그에 맞춰 코드를 수정하였다. 또한 AI 로 부터 깨달은 내용 중 가장 흥미로웠던 점은 이중 for 문에서 $j = i + 1$ 로 시작하면 자기 자신과의 비교와 중복 비교를 피할 수 있다는 점이었다.